

Seguridad de Growth Horizon - explicado facil

Este documento explica, sin tecnicismos, que protecciones tiene la aplicacion y por que existen. Cada seccion tiene una comparacion de la vida real para que quede claro sin saber de programacion.

1. Las contraseñas nunca se guardan "en texto plano"

Que pasaria si no hicieramos esto: si alguien roba la base de datos, tendria la lista completa de contraseñas de todos los usuarios, listas para usar en cualquier otra pagina (mucha gente repite contraseña en varios sitios).

Que hacemos: cuando te registras, tu contraseña pasa por una funcion llamada bcrypt que la convierte en una mezcla de letras y numeros sin sentido (un "hash"), y esa mezcla no se puede deshacer. Es como meter tu contraseña en una licuadora: puedes comparar si el batido de hoy es igual al de ayer, pero no puedes sacar la fruta original de un batido ya hecho.

Cuando inicias sesion, el sistema licua la contraseña que escribiste y compara el resultado con el batido guardado. Si coinciden, entras. Nadie - ni siquiera un desarrollador con acceso a la base de datos - puede ver tu contraseña real.

Ademas, exigimos que sea fuerte: minimo 8 caracteres, con al menos una mayuscula y un numero. Es como pedir que el candado de tu bicicleta no sea "0000".

2. Si alguien intenta adivinar tu contraseña a la fuerza, lo bloqueamos

El problema que evitamos: un atacante podria escribir un programa que pruebe miles de contraseñas por segundo contra tu cuenta ("fuerza bruta"), como alguien probando todas las combinaciones de una caja fuerte.

Que hacemos: es igual que un cajero automatico. Si fallas tu clave 5 veces en 15 minutos, la cuenta se congela durante 10 minutos, aunque la sexta vez escribas la clave correcta. Esto vuelve inutil el intentar adivinar por fuerza bruta, porque el atacante solo puede probar 5 claves cada 15 minutos - tardaria anos en adivinar algo.

Cada intento fallido queda anotado (quien, cuando, desde que direccion), asi que si alguien intenta forzar una cuenta, queda un rastro.

3. Nadie puede "colarse" leyendo o cambiando la direccion en el navegador

El problema que evitamos: imagina un hotel donde la llave de tu habitacion tambien abriera la del vecino con solo cambiar el numero de puerta. Eso seria un desastre.

Que hacemos: cada empresa registrada es como una habitacion de hotel con su propia llave. Cuando un

usuario de la Empresa A pide ver una evaluacion, el sistema siempre revisa: "esta evaluacion es realmente de tu empresa?". Si no lo es, aunque el usuario adivine o escriba el numero exacto en la barra de direcciones, el sistema responde con un "Acceso denegado" (error 403). Lo probamos a proposito creando dos empresas y tratando de que una viera los datos de la otra: funciono como se esperaba.

4. No todos pueden hacer lo mismo dentro de la plataforma

Comparacion: en una empresa real, el conserje no tiene la llave maestra de todas las oficinas, y el dueño no necesita pedir permiso para entrar a ninguna. Aqui pasa lo mismo con dos "llaves":

- SUPERADMIN (el equipo de Growth Horizon): la "llave maestra". Puede ver y administrar todas las empresas, el cuestionario, las recomendaciones, todo.
 - EMPRESA (quien se registra): solo tiene la "llave" de su propia empresa. Puede responder su diagnostico y ver sus propios resultados, nada mas.
-

5. Nadie puede "hackear" la base de datos escribiendo cosas raras en el buscador de login

El problema que evitamos: hay un ataque viejo llamado "inyeccion SQL" donde, en vez de escribir un correo normal, alguien escribe algo como `' ; DROP TABLE usuarios; --` esperando que el sistema lo confunda con una orden y borre toda la tabla de usuarios.

Que hacemos: la aplicacion nunca arma las preguntas a la base de datos pegando texto a mano. Usa una herramienta (el ORM SQLAlchemy) que siempre trata lo que escribes como un dato, nunca como una orden. Es la diferencia entre decirle a un bibliotecario "busca el libro llamado Quema la biblioteca" (el bibliotecario busca ese titulo raro y no encuentra nada) contra darle una orden real de quemar la biblioteca. Se probó enviando varios de estos ataques clasicos por el formulario de login: ninguno funciono, y la tabla de usuarios siguio intacta.

6. Nadie puede meter codigo espia en lo que ve otro usuario

El problema que evitamos: el ataque "XSS" ocurre cuando alguien escribe, por ejemplo, el nombre de su empresa como `<script>robame_la_sesion()</script>`, esperando que ese codigo se ejecute en la pantalla de otra persona que vea ese nombre (por ejemplo, un administrador viendo la lista de empresas).

Que hacemos: cada vez que la pagina muestra un dato que escribio un usuario (nombre, correo, etc.), el sistema lo convierte automaticamente en texto "inofensivo" antes de mostrarlo. Si alguien escribe `<script>...</script>` como nombre de empresa, en la pantalla se ve literalmente el texto `<script>...</script>`, como si fuera una foto de ese texto, no un trozo de pagina web real. Nunca se apaga esta proteccion en

ningun formulario del proyecto.

7. Que se anota y se guarda (auditoria)

Piensa en esto como las camaras de seguridad de un edificio: no evitan que algo pase, pero dejan constancia de quien entro, cuando, y que intento hacer. La aplicacion anota automaticamente:

- Cada inicio de sesion exitoso o fallido (con fecha, hora e IP)
 - Cada bloqueo por intentos fallidos
 - Cada registro de una nueva empresa
 - Cada vez que se crea, edita o borra una empresa o un sector
 - Alertas especiales si alguien intenta (sin exito) entrar a una cuenta SUPERADMIN
-

8. Lo que TODAVIA no esta protegido (pendiente)

Para ser honestos: encontramos una cosa que no esta protegida todavia, aunque en la configuracion parezca que si.

CSRF (falsificacion de peticiones entre sitios): imagina que estas con la sesion iniciada en el banco, y sin darte cuenta visitas una pagina maliciosa que, escondida, le manda una orden a tu banco como si fueras tu ("transferir \$1000"), aprovechando que tu navegador todavia tiene tu sesion abierta. La proteccion normal contra esto es pedir, ademas de la sesion, un "codigo secreto de un solo uso" en cada formulario que cambia algo (crear, editar, borrar). Ahora mismo, ese codigo no se pide en ningun formulario, aunque en la configuracion hay una bandera que dice `WTF_CSRF_ENABLED = True` - esa bandera sola no hace nada si no se activa la pieza que realmente la aplica, y esa pieza no esta activada.

Esto se confirmo con una prueba automatica (ver `tests/test_seguridad.py`, seccion "EXTRA"): se mando una peticion que cambia datos sin ningun codigo de seguridad, y la aplicacion la acepto igual. Es un pendiente real, no solo teoria.

9. Glosario para quien quiera profundizar un poco mas

Palabra	En una frase
Hash	El "batido" irreversible en el que se convierte una contrasena.
Bcrypt	La herramienta que hace ese "batido" de forma segura y lenta a proposito (para que probar millones de contrasenas sea muy costoso).
Fuerza bruta	Probar muchisimas contrasenas seguidas hasta acertar una.
SQL Injection	Intentar colar ordenes a la base de datos disfrazadas de texto normal.
XSS	Meter codigo espia disfrazado de texto normal para que se ejecute en la pantalla de otra persona.
CSRF	Enganar tu sesion ya abierta para que haga algo sin que tu lo pidas.

Palabra	En una frase
ORM	La capa que traduce el código de la aplicación a preguntas seguras para la base de datos.
Multi-tenant	Que varias empresas comparten la misma aplicación pero cada una solo ve lo suyo.
Auditoria	El registro (como cámaras de seguridad) de quien hizo que y cuando.
Rol / permiso	Que tan "maestra" es tu llave dentro de la plataforma.

10. Como comprobarlo tu mismo

No hace falta confiar en la palabra de nadie: hay un script que revisa todo lo de arriba (menos el glosario) automáticamente, sin tocar la base de datos real:

```
python tests/test_seguridad.py
```

Cada línea imprime PASS (funciona como debe) o FAIL (algo se rompió).